

Convolutional Neural Network (CNN)



Run in Google Colab (<https://colab.research.google.com/github/tensorflow/docs/blob/master/site/en/tutorials/images/cnn.ipynb>)



View source on GitHub (<https://github.com/tensorflow/docs/blob/master/site/en/tutorials/images/cnn.ipynb>)



Download notebook (https://storage.googleapis.com/tensorflow_docs/docs/site/en/tutorials/images/cnn.ipynb)

This tutorial demonstrates training a simple Convolutional Neural Network

(https://developers.google.com/machine-learning/glossary/#convolutional_neural_network) (CNN) to classify CIFAR images (<https://www.cs.toronto.edu/%7Ekriz/cifar.html>). Because this tutorial uses the Keras Sequential API (<https://www.tensorflow.org/guide/keras/overview>), creating and training your model will take just a few lines of code.

Import TensorFlow

```
import tensorflow as tf

from tensorflow.keras import datasets, layers, models
import matplotlib.pyplot as plt
```

```
2024-08-16 03:19:29.442443: E external/local_xla/xla/stream_executor/cuda/cuda_fft.cc:48
2024-08-16 03:19:29.463718: E external/local_xla/xla/stream_executor/cuda/cuda_dnn.cc:84
2024-08-16 03:19:29.470590: E external/local_xla/xla/stream_executor/cuda/cuda_blas.cc:111
```

Download and prepare the CIFAR10 dataset

The CIFAR10 dataset contains 60,000 color images in 10 classes, with 6,000 images in each class. The dataset is divided into 50,000 training images and 10,000 testing images. The classes are mutually exclusive and there is no overlap between them.

```
(train_images, train_labels), (test_images, test_labels) = datasets.cifar10.load_data()

# Normalize pixel values to be between 0 and 1
train_images, test_images = train_images / 255.0, test_images / 255.0
```

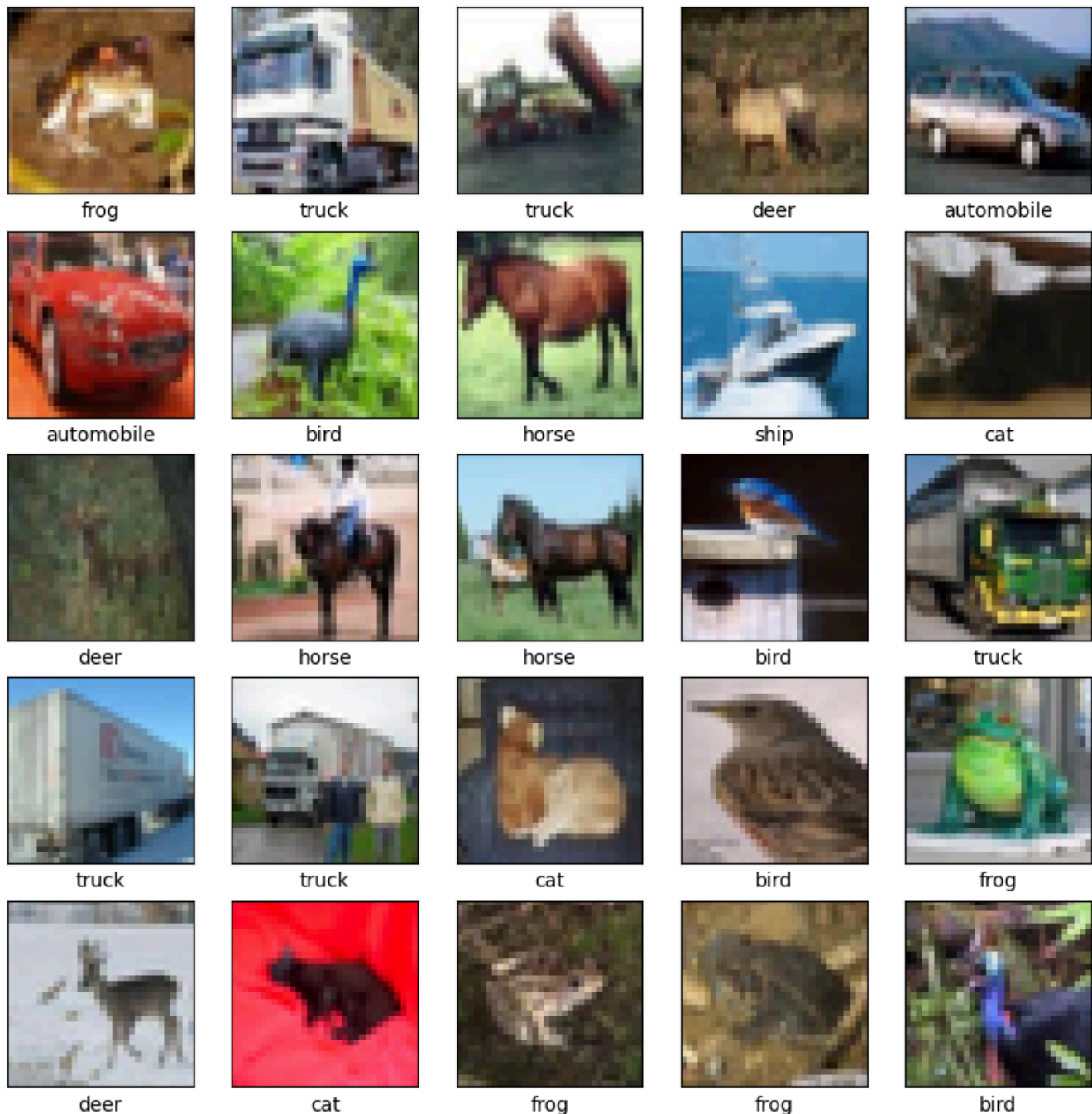
Downloading data from <https://www.cs.toronto.edu/~kriz/cifar-10-python.tar.gz>
170498071/170498071 ————— 4s 0us/step

Verify the data

To verify that the dataset looks correct, let's plot the first 25 images from the training set and display the class name below each image:

```
class_names = ['airplane', 'automobile', 'bird', 'cat', 'deer',
               'dog', 'frog', 'horse', 'ship', 'truck']

plt.figure(figsize=(10,10))
for i in range(25):
    plt.subplot(5,5,i+1)
    plt.xticks([])
    plt.yticks([])
    plt.grid(False)
    plt.imshow(train_images[i])
    # The CIFAR labels happen to be arrays,
    # which is why you need the extra index
    plt.xlabel(class_names[train_labels[i][0]])
plt.show()
```



Create the convolutional base

The 6 lines of code below define the convolutional base using a common pattern: a stack of [Conv2D](https://www.tensorflow.org/api_docs/python/tf/keras/layers/Conv2D) (https://www.tensorflow.org/api_docs/python/tf/keras/layers/Conv2D) and [MaxPooling2D](https://www.tensorflow.org/api_docs/python/tf/keras/layers/MaxPool2D) (https://www.tensorflow.org/api_docs/python/tf/keras/layers/MaxPool2D) layers.

As input, a CNN takes tensors of shape (image_height, image_width, color_channels), ignoring the batch size. If you are new to these dimensions, color_channels refers to (R,G,B). In this

example, you will configure your CNN to process inputs of shape (32, 32, 3), which is the format of CIFAR images. You can do this by passing the argument `input_shape` to your first layer.

```
model = models.Sequential()
model.add(layers.Conv2D(32, (3, 3), activation='relu', input_shape=(32, 32, 3)))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
model.add(layers.MaxPooling2D((2, 2)))
model.add(layers.Conv2D(64, (3, 3), activation='relu'))
```

```
/tmpfs/src/tf_docs_env/lib/python3.9/site-packages/keras/src/layers/convolutional/base_
super().__init__(activity_regularizer=activity_regularizer, **kwargs)
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1723778380.352952 134940 cuda_executor.cc:1015] successful NUMA node read
I0000 00:00:1723778380.356800 134940 cuda_executor.cc:1015] successful NUMA node read
I0000 00:00:1723778380.360496 134940 cuda_executor.cc:1015] successful NUMA node read
I0000 00:00:1723778380.364197 134940 cuda_executor.cc:1015] successful NUMA node read
I0000 00:00:1723778380.375962 134940 cuda_executor.cc:1015] successful NUMA node read
I0000 00:00:1723778380.379580 134940 cuda_executor.cc:1015] successful NUMA node read
I0000 00:00:1723778380.382914 134940 cuda_executor.cc:1015] successful NUMA node read
I0000 00:00:1723778380.386363 134940 cuda_executor.cc:1015] successful NUMA node read
I0000 00:00:1723778380.389687 134940 cuda_executor.cc:1015] successful NUMA node read
I0000 00:00:1723778380.393093 134940 cuda_executor.cc:1015] successful NUMA node read
```

Let's display the architecture of your model so far:

```
model.summary()
```

Above, you can see that the output of every Conv2D and MaxPooling2D layer is a 3D tensor of shape (height, width, channels). The width and height dimensions tend to shrink as you go deeper in the network. The number of output channels for each Conv2D layer is controlled by the first argument (e.g., 32 or 64). Typically, as the width and height shrink, you can afford (computationally) to add more output channels in each Conv2D layer.

Add Dense layers on top

To complete the model, you will feed the last output tensor from the convolutional base (of shape (4, 4, 64)) into one or more Dense layers to perform classification. Dense layers take vectors as input (which are 1D), while the current output is a 3D tensor. First, you will flatten (or unroll) the 3D output to 1D, then add one or more Dense layers on top. CIFAR has 10 output classes, so you use a final Dense layer with 10 outputs.

```
model.add(layers.Flatten())  
model.add(layers.Dense(64, activation='relu'))  
model.add(layers.Dense(10))
```

Here's the complete architecture of your model:

```
model.summary()
```

The network summary shows that (4, 4, 64) outputs were flattened into vectors of shape (1024) before going through two Dense layers.

Compile and train the model

```
model.compile(optimizer='adam',
              loss=tf.keras.losses.SparseCategoricalCrossentropy(from_logits=True),
              metrics=['accuracy'])

history = model.fit(train_images, train_labels, epochs=10,
                    validation_data=(test_images, test_labels))
```

Epoch 1/10

```
WARNING: All log messages before absl::InitializeLog() is called are written to STDERR
I0000 00:00:1723778386.322623 135108 service.cc:146] XLA service 0x7f254c005a30 initi
I0000 00:00:1723778386.322659 135108 service.cc:154] StreamExecutor device (0): Tes
I0000 00:00:1723778386.322663 135108 service.cc:154] StreamExecutor device (1): Tes
I0000 00:00:1723778386.322667 135108 service.cc:154] StreamExecutor device (2): Tes
I0000 00:00:1723778386.322670 135108 service.cc:154] StreamExecutor device (3): Tes
70/1563 ─────────────────── 3s 2ms/step - accuracy: 0.1470 - loss: 2.2585
I0000 00:00:1723778388.120198 135108 device_compiler.h:188] Compiled cluster using XLA
1563/1563 ─────────────────── 10s 4ms/step - accuracy: 0.3569 - loss: 1.7377 - val_ac
```

Epoch 2/10

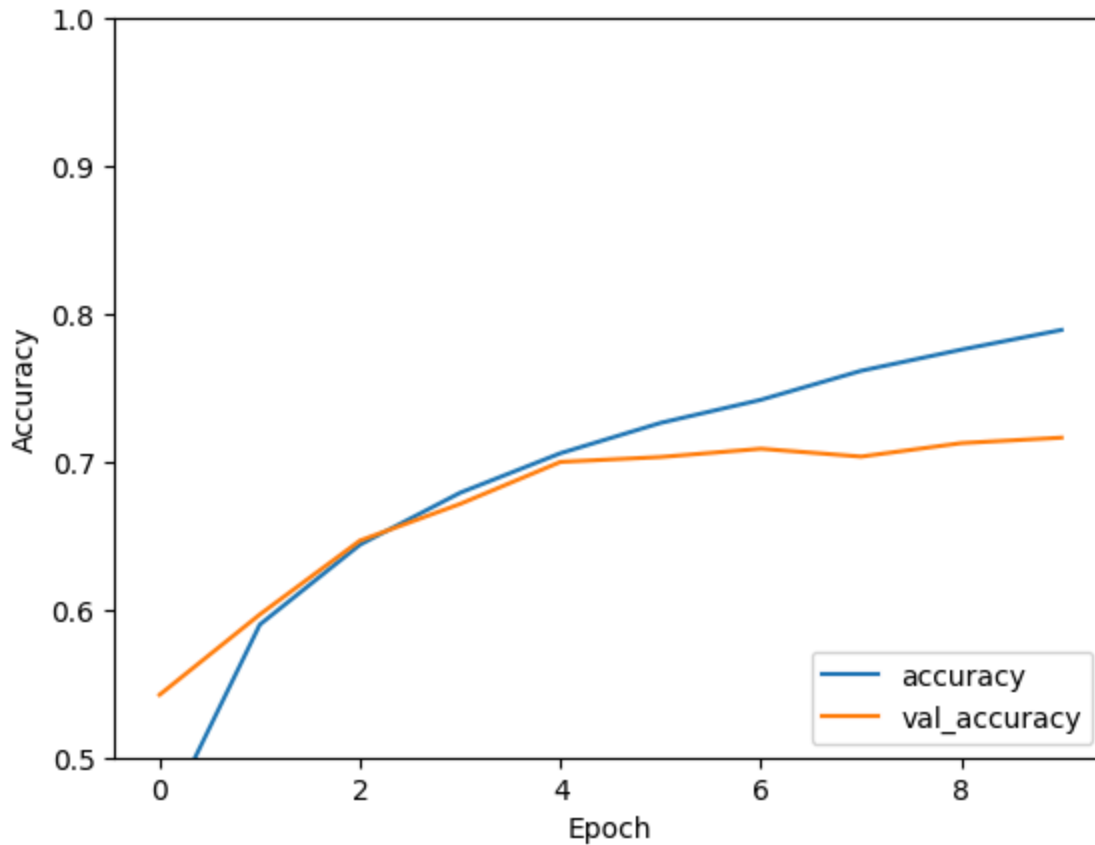
1563/1563 — 4s 2ms/step - accuracy: 0.5721 - loss: 1.1966 - val_accu

Evaluate the model

```
plt.plot(history.history['accuracy'], label='accuracy')
plt.plot(history.history['val_accuracy'], label = 'val_accuracy')
plt.xlabel('Epoch')
plt.ylabel('Accuracy')
plt.ylim([0.5, 1])
plt.legend(loc='lower right')
```

```
test_loss, test_acc = model.evaluate(test_images, test_labels, verbose=2)
```

313/313 - 0s - 2ms/step - accuracy: 0.7163 - loss: 0.8550




```
print(test_acc)
```

```
0.7163000106811523
```

Your simple CNN has achieved a test accuracy of over 70%. Not bad for a few lines of code! For another CNN style, check out the [TensorFlow 2 quickstart for experts](https://www.tensorflow.org/tutorials/quickstart/advanced) (<https://www.tensorflow.org/tutorials/quickstart/advanced>) example that uses the Keras subclassing API and [tf.GradientTape](https://www.tensorflow.org/api_docs/python/tf/GradientTape) (https://www.tensorflow.org/api_docs/python/tf/GradientTape).

Except as otherwise noted, the content of this page is licensed under the [Creative Commons Attribution 4.0 License](https://creativecommons.org/licenses/by/4.0/) (<https://creativecommons.org/licenses/by/4.0/>), and code samples are licensed under the [Apache 2.0 License](https://www.apache.org/licenses/LICENSE-2.0) (<https://www.apache.org/licenses/LICENSE-2.0>). For details, see the [Google Developers Site Policies](https://developers.google.com/site-policies) (<https://developers.google.com/site-policies>). Java is a registered trademark of Oracle and/or its affiliates.

Last updated 2024-08-16 UTC.